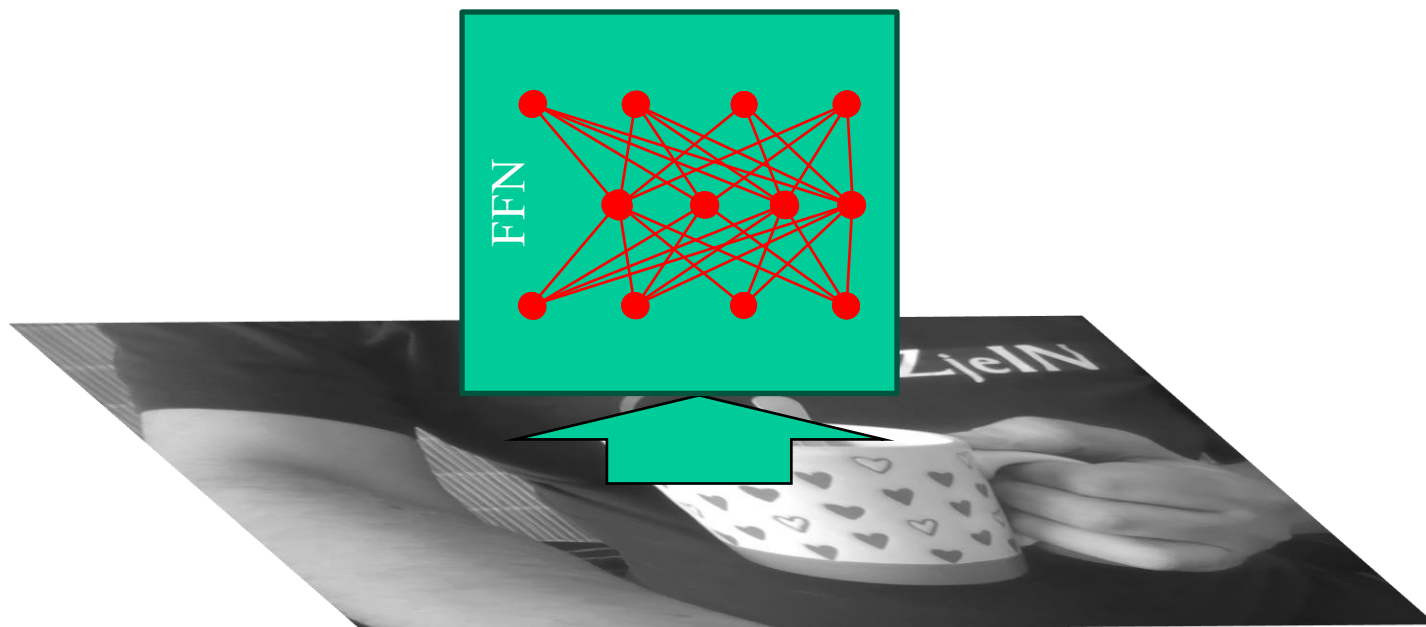


Konvolučný autokóder a klasifikátor, 5.11.2025, 15:00-17:00

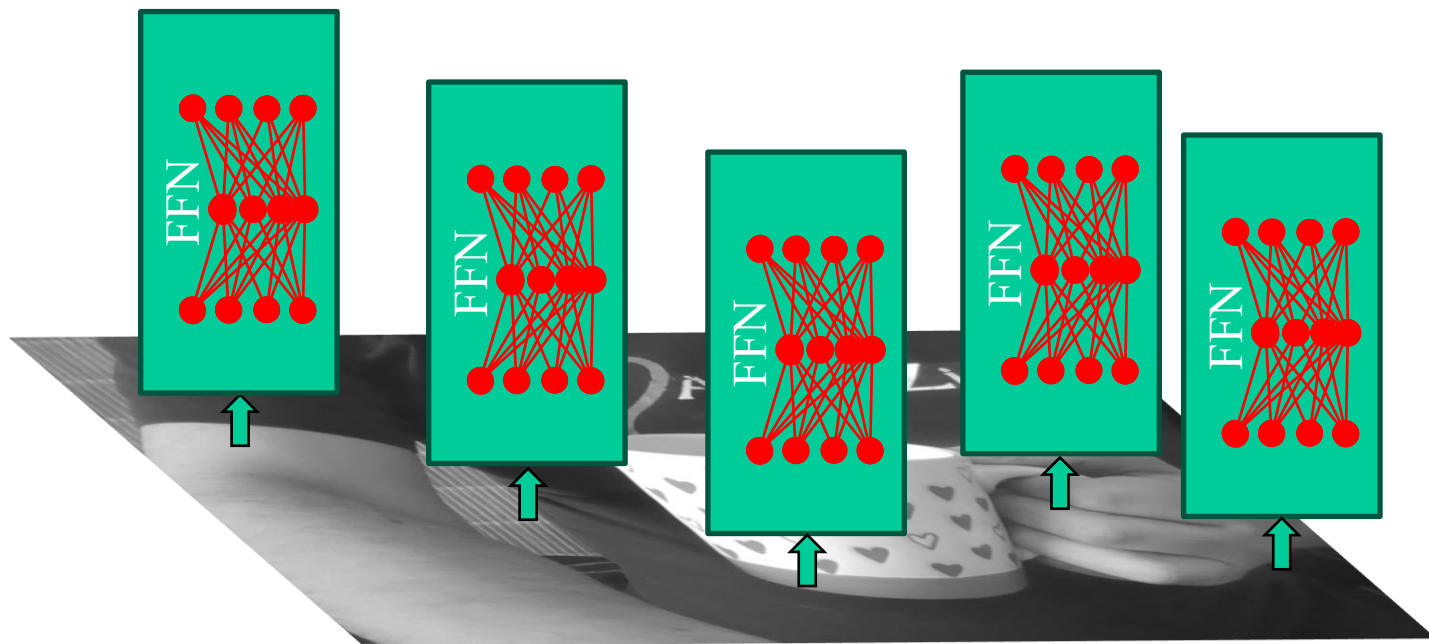
Andrej Lúčny

Autokóder. Príznakové vektory, latentný priestor. Kóder, klasifikátor obrazov. Dekóder, generátor obrazov. Chrbticové siete premieňajúce obraz na príznaky (VGG, ResNet).

<https://github.com/andylucny/OAI>



Hoci Perceptron (Feed Forward Network) je univerzálny aproximátor, v praxi nie je spracovanie obrazu priamo použiteľný



Obráz ale môžeme spracúvať menšími paralelne a lokálne operujúcimi perceptrónmi, ktoré zdieľajú váhy (kernely 1×1) a prípadne aj spolupracujú so susedmi (kernely 3×3) ...

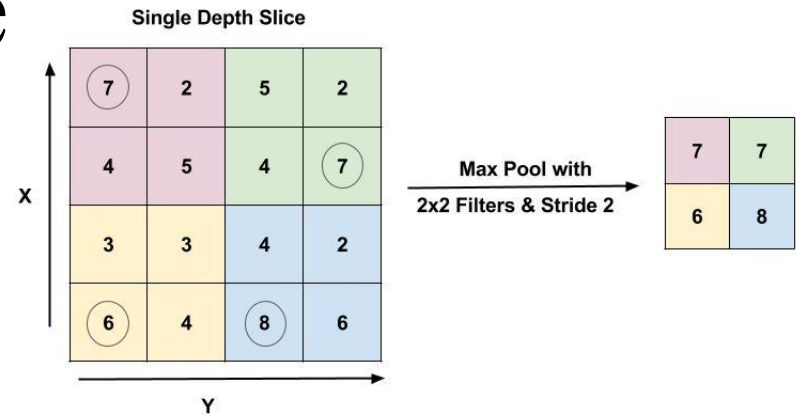
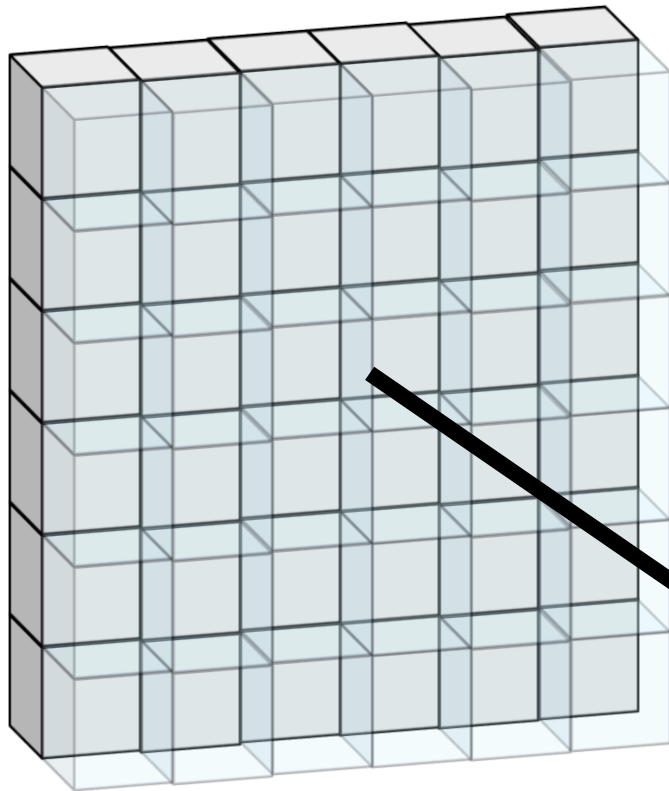


... čo zodpovedá dvom za seba radeným blokom konvolučných vrstiev, ktoré premieňajú farebné kanály obrazu na príznaky a tie na ďalšie príznaky

Redukcia dimenzie

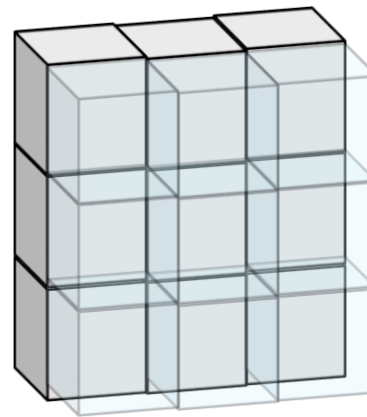
- Náš plán bol spracovať obraz (čo je vstup veľkej dimenzie) na niečo oveľa menšej dimenzie, avšak bez straty informačnej hodnoty
- Keď to bude dostatočne malé, perceptron si s tým potom už poradí

Redukcia dimenzie na báze maxima

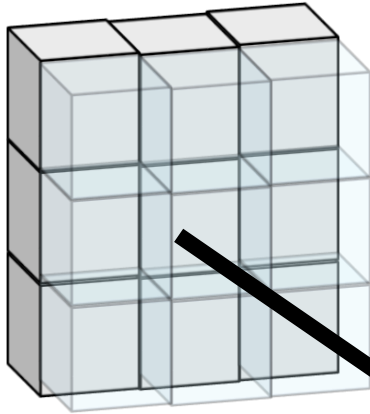


zlučovanie 2x2 pixelov s krokom 2
a ich nahradenie maximom

MaxPooling2D 2x2 stride=2



Espansione dimensione



Nearest Neighbor

1	2
3	4

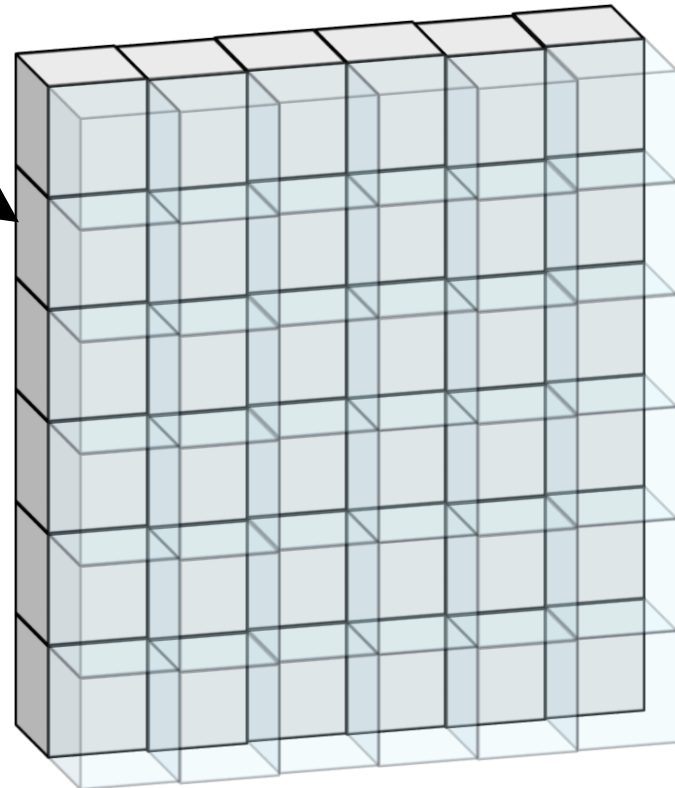


1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

Input: 2 x 2

Output: 4 x 4

Upsampling2D 2x2 stride=2



Autokóder

Pracujeme s datasetom neanotovaných obrázkov



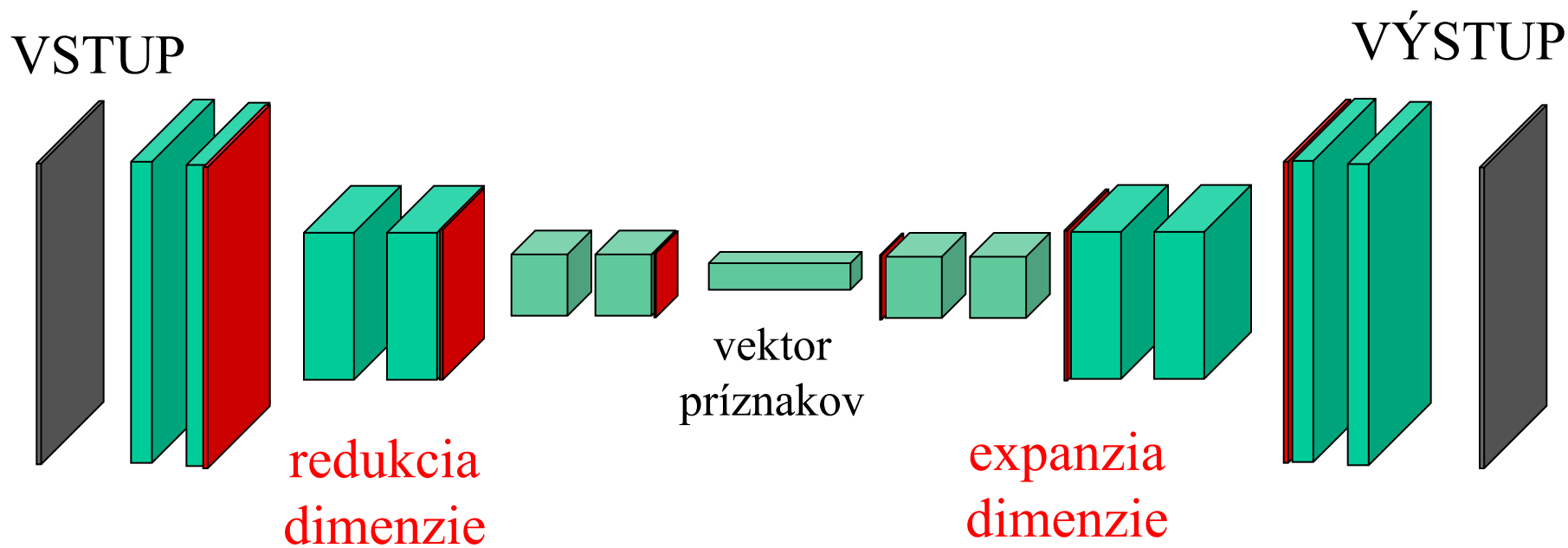
obraz transformujeme na príznaky malej dimenzie



Dobré príznaky vieme nielen zakódovať (extrahovať) ale vieme z nich aj dekodovať (generovať) pôvodný obraz

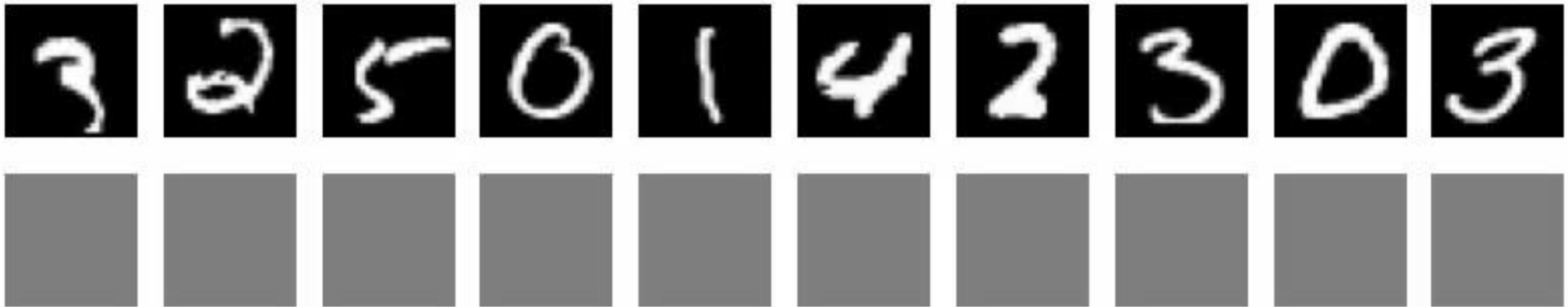
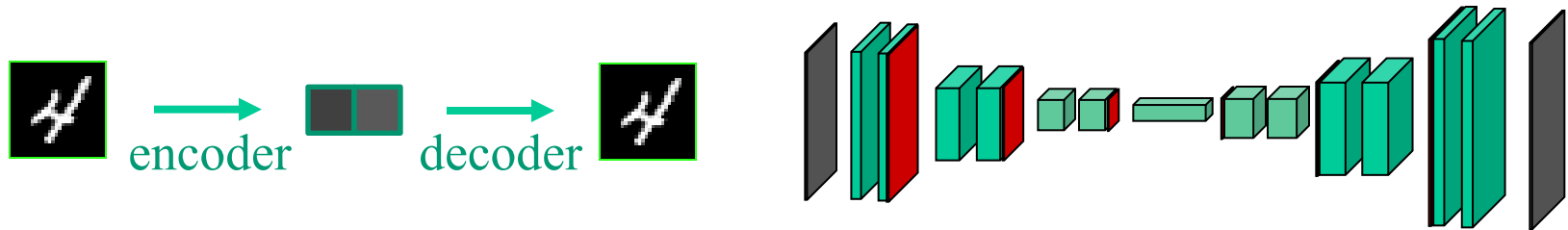


Konvolučný autokóder



chceme, aby:
 $VSTUP = VÝSTUP$

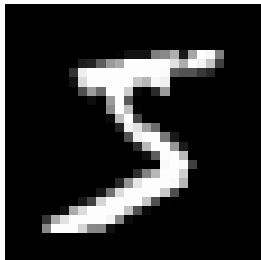
Autoencoder



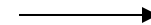
Autoencoder

LATENT SPACE

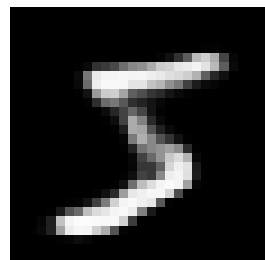
INPUT



```
tensor([0.5128, 0.4216, 0.4599, 0.5000, 0.4729, 0.3759, 0.4845, 0.4800, 0.4545,  
0.4549, 0.3755, 0.3943, 0.4592, 0.4887, 0.3720, 0.4864, 0.4326, 0.3576,  
0.2825, 0.4351, 0.6167, 0.3217, 0.3943, 0.5935, 0.3106, 0.3975, 0.6174,  
0.5864, 0.4706, 0.3994, 0.6045, 0.4558, 0.3407, 0.5280, 0.4200, 0.3513,  
0.4281, 0.4669, 0.4538, 0.4297, 0.5300, 0.4694, 0.4335, 0.4635, 0.4034,  
0.5179, 0.3048, 0.2911, 0.5669, 0.3968, 0.2865, 0.5514, 0.3497, 0.3256,  
0.4478, 0.5840, 0.3743, 0.4684, 0.5981, 0.3716, 0.4457, 0.5995, 0.4222,  
0.3256, 0.3229, 0.5112, 0.3323, 0.3461, 0.4971, 0.3396, 0.3261, 0.4653])
```



OUTPUT

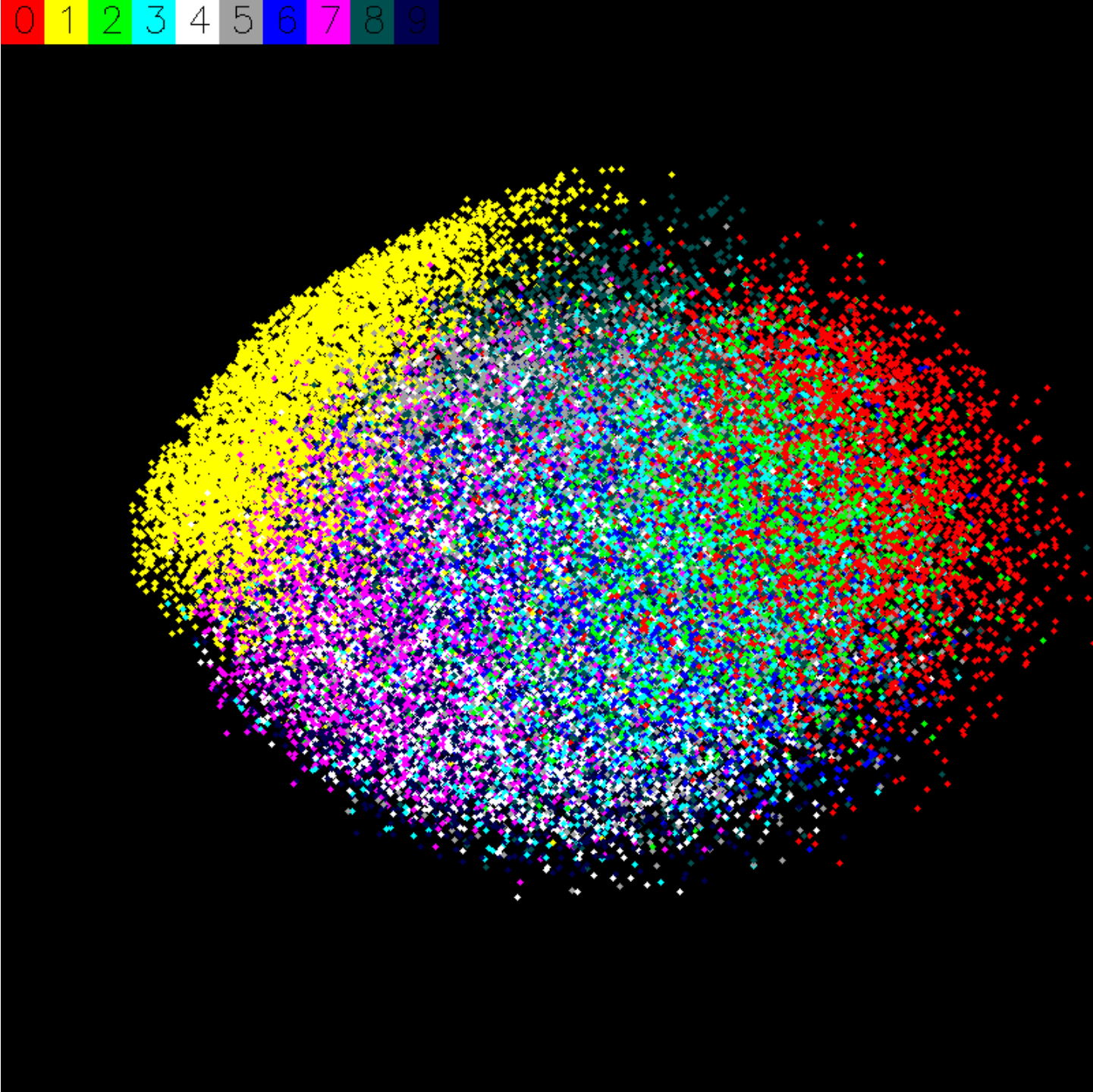


feature vector (príznakový vektor)

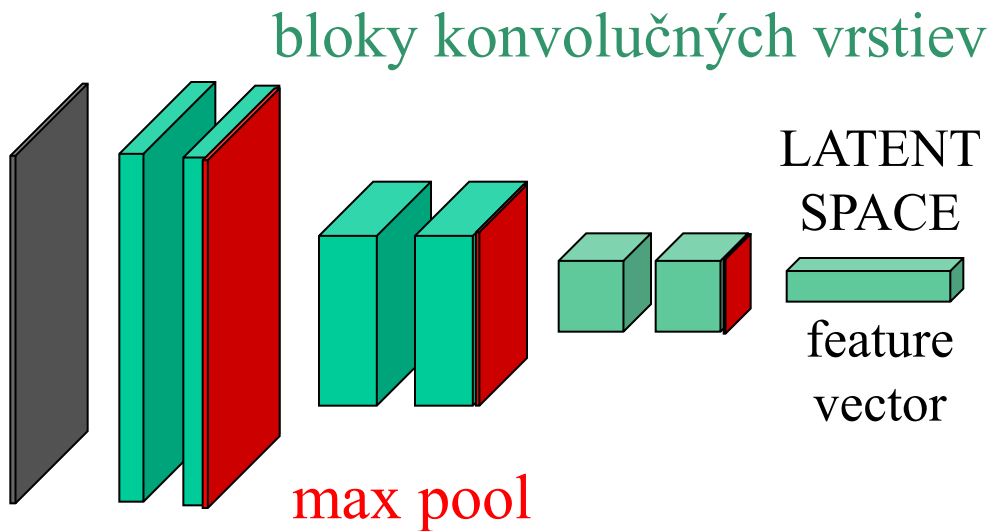
Základné vlastnosti príznakov

- Príznakové vektory možno zobrazit' ako body v latentnom priestore
- Hoci len konečný počet príznakov zodpovedá vzorkám z datasetu, všetky body latentného priestoru zodpovedajú nejakej inštancii dát
- Latentný priestor nemá diery (not sparse) a je plynulý (uniformly continuous)

LATENT SPACE

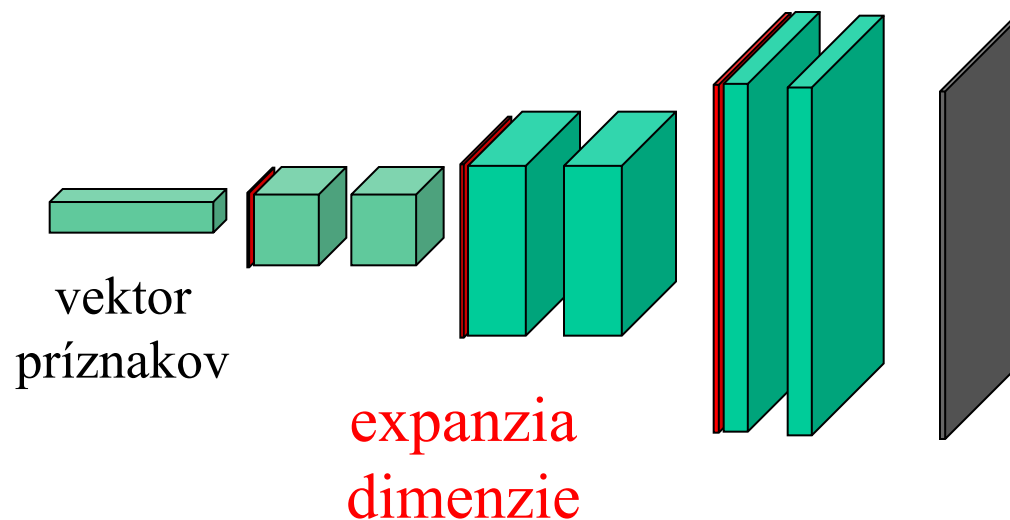


Encoder (Extractor)



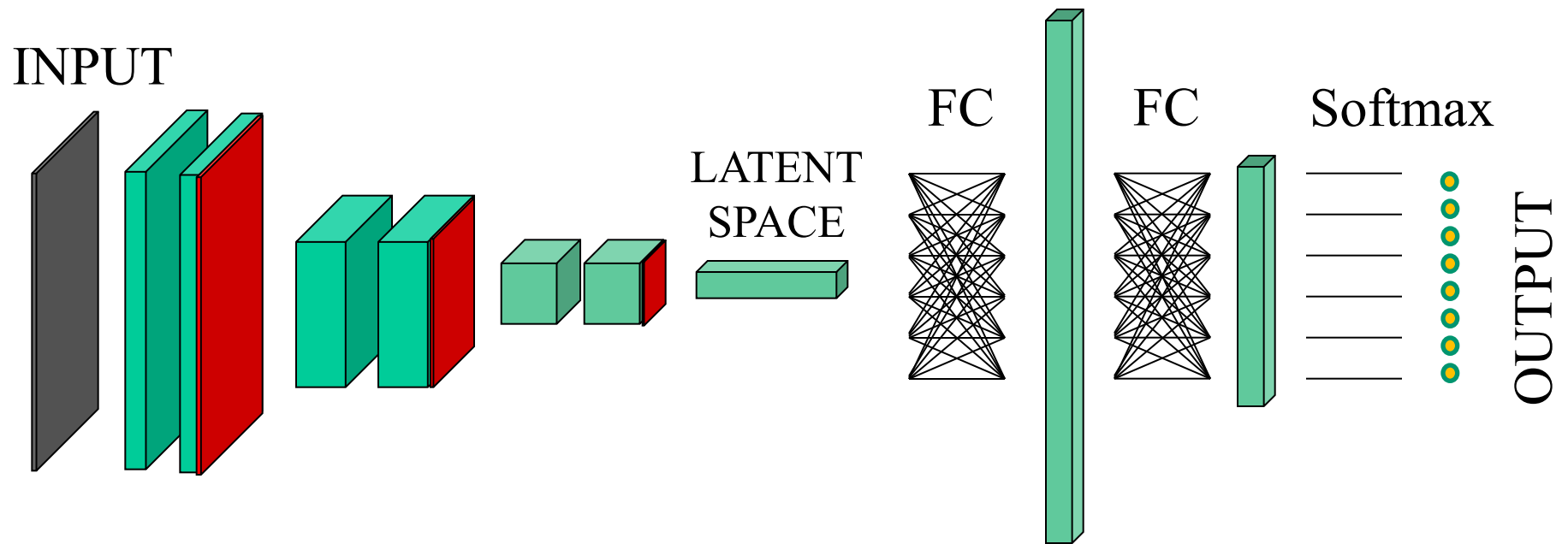
Encoder (prvá polovica autokódera) transformuje obraz na príznaky (bod v latentnom priestore) ...

Decoder (Generator)



Decoder (druhá polovica autokódera)
transformuje (akékoľvek) príznavy na obraz

Klasifikátor



logit
Softmax mení čokoľvek na pravdepodobnosti

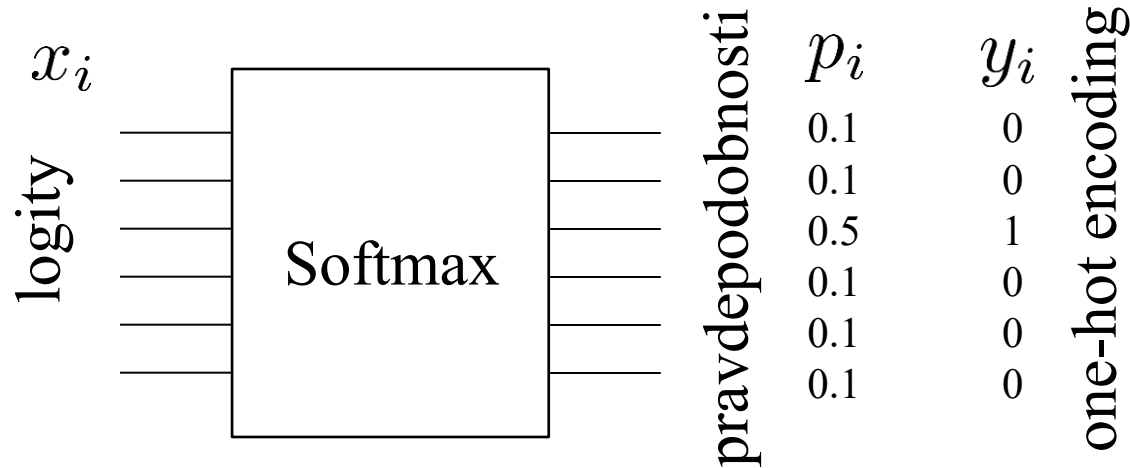
$$\text{softmax}_i(x) = \frac{e^{x_i}}{\sum_k e^{x_k}}$$

```
def softmax(x):  
    e_x = np.exp(x - np.max(x))  
    return e_x / e_x.sum(axis=0)
```

Softmax má zmysel len keď sa kategórie navzájom vylučujú.
Pokiaľ máme napr. koleso aj bicykel, používame Sigmoid

Cross Entropy Loss

$$\text{loss}(x, y) = - \sum_i y_i \log \frac{e^{x_i}}{\sum_j e^{x_j}}$$



gradienty

$$\frac{\partial L}{\partial x_i} = \begin{cases} p_i & \text{when } y_i = 0 \\ p_i - 1 & \text{when } y_i = 1 \end{cases}$$

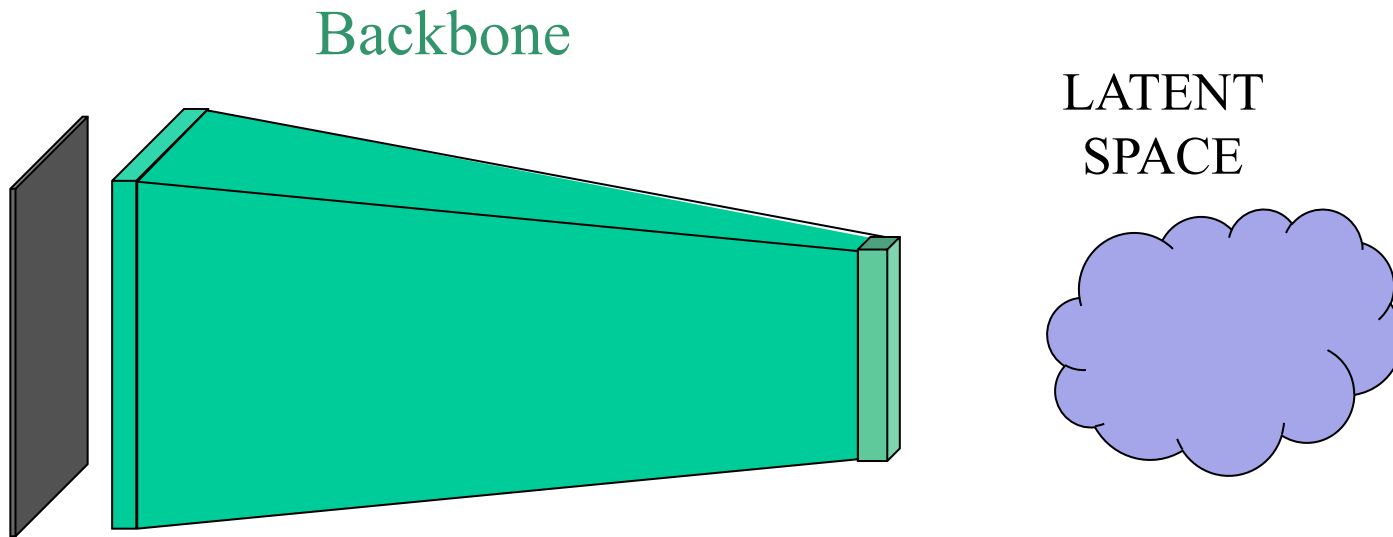
Dôležitý princíp

- Keď si vieme zdôvodniť ako nejakú DNN urobiť postupne a po častiach (spravíme autoencoder, odrežeme dekóder, pripojíme perceptron, ...), môžeme ju natrénovať priamo v konečnej podobe, naraz a v celku (end-to-end system)

Chrbticové siete (Backbones)

- Rýchlejšie však môže byť vziať vhodný hotový extraktor príznakov, na trénovať k nemu napríklad perceptron a až potom celý model pretrénovať (Fine tuning = jemné doladenie)
- Backbones: VGG, ResNet, MobileNet, ...
- ZOO, každá backbone architektúra má k dispozícii viacero predtrénovaných modelov (torchvision)

Backbone

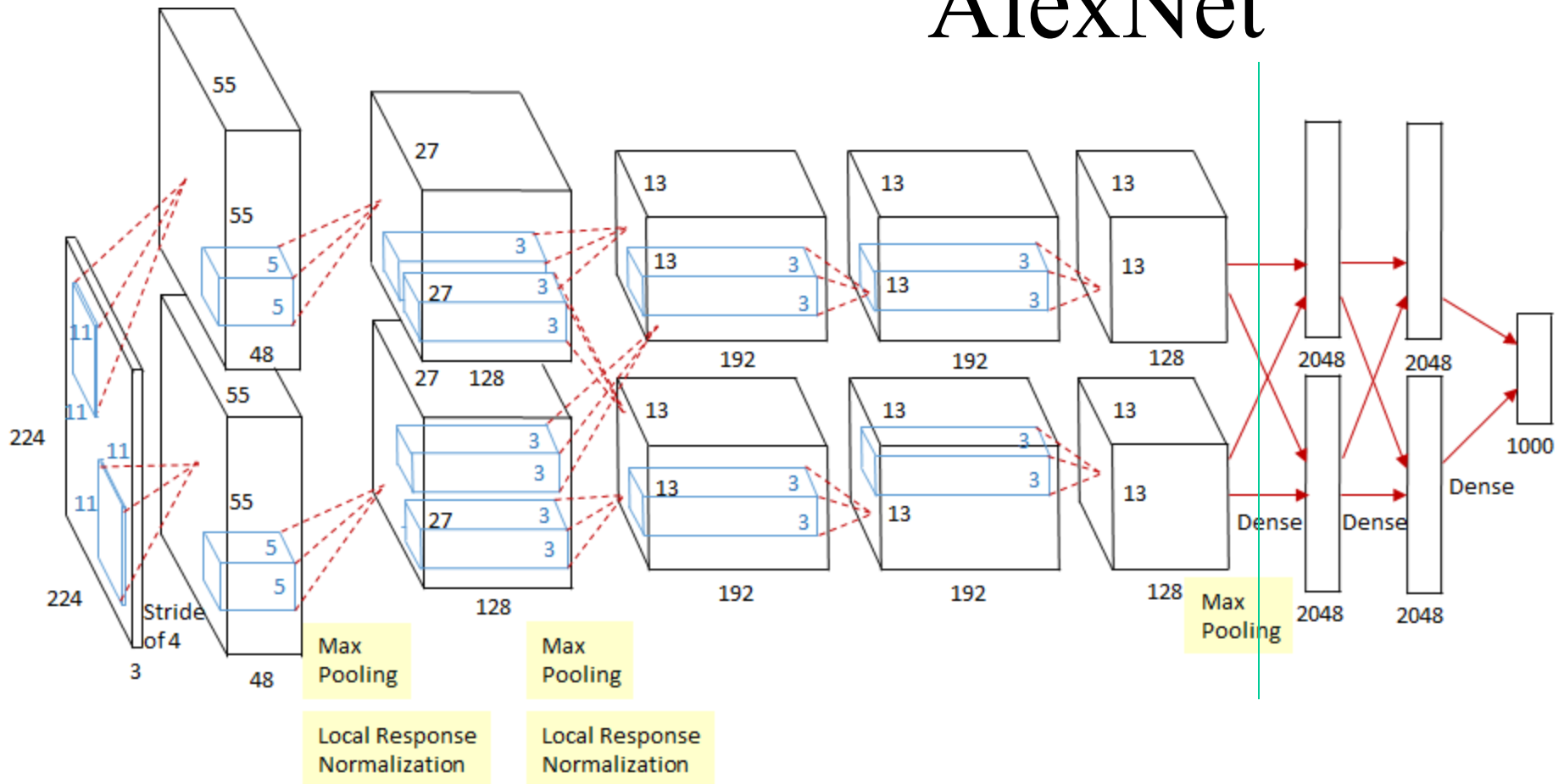


- Časť siete, ktorá obraz premení na príznaky

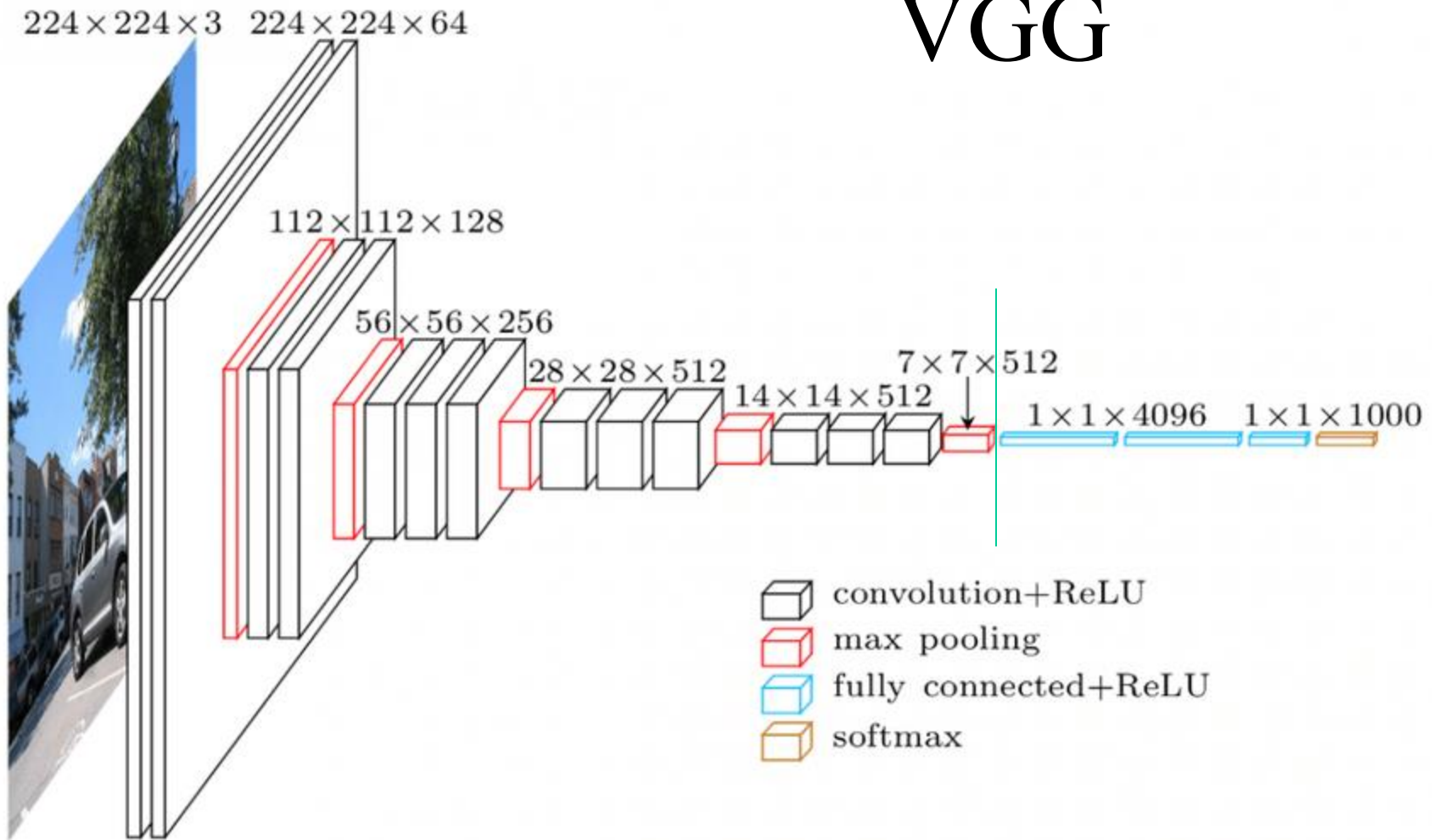
Backbones

- AlexNet
- VGG (VGG16, VGG19)
- ResNet (ResNet50, ResNet18)
- DarkNet
- Inception (Inception v3)
- Xception
- MobileNet

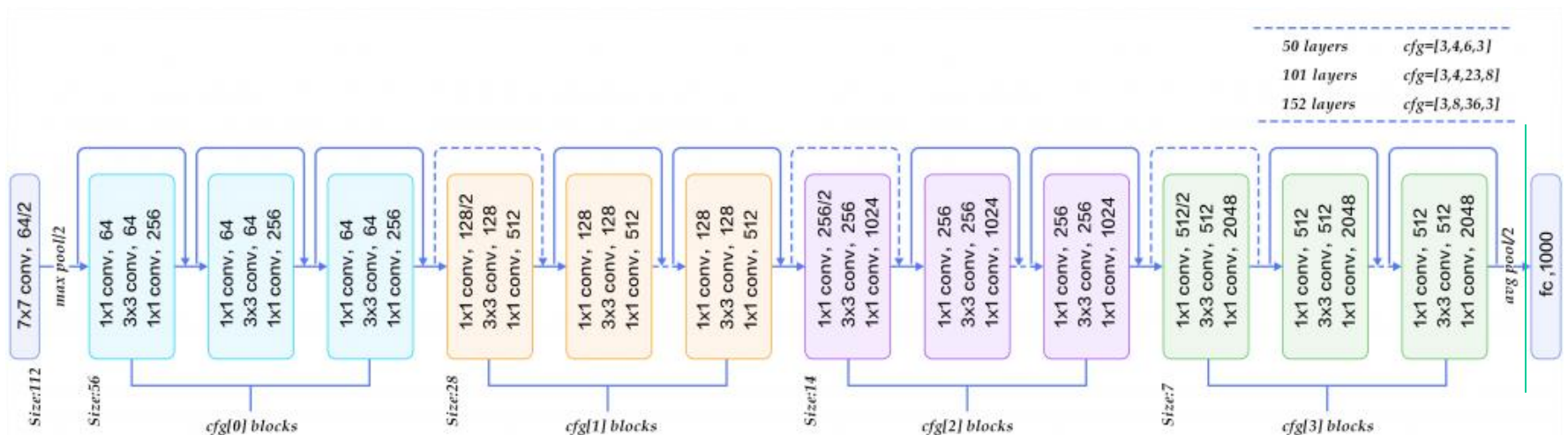
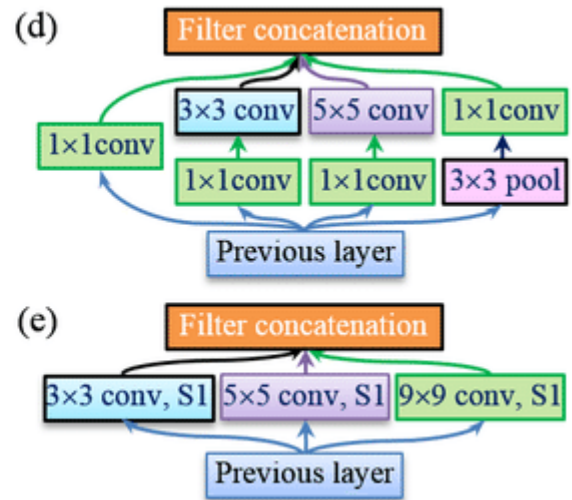
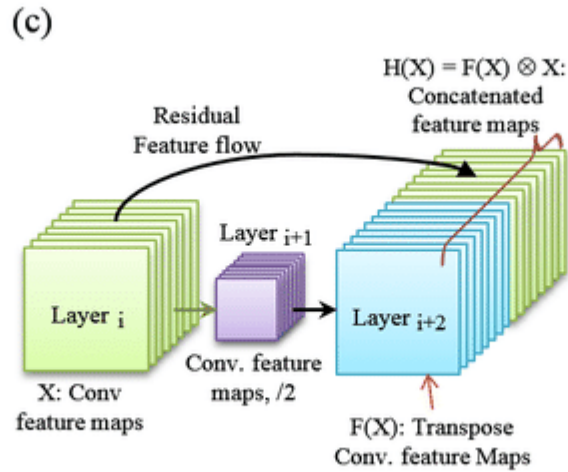
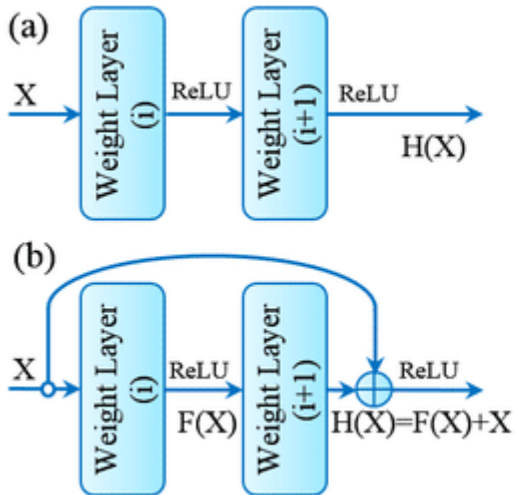
AlexNet



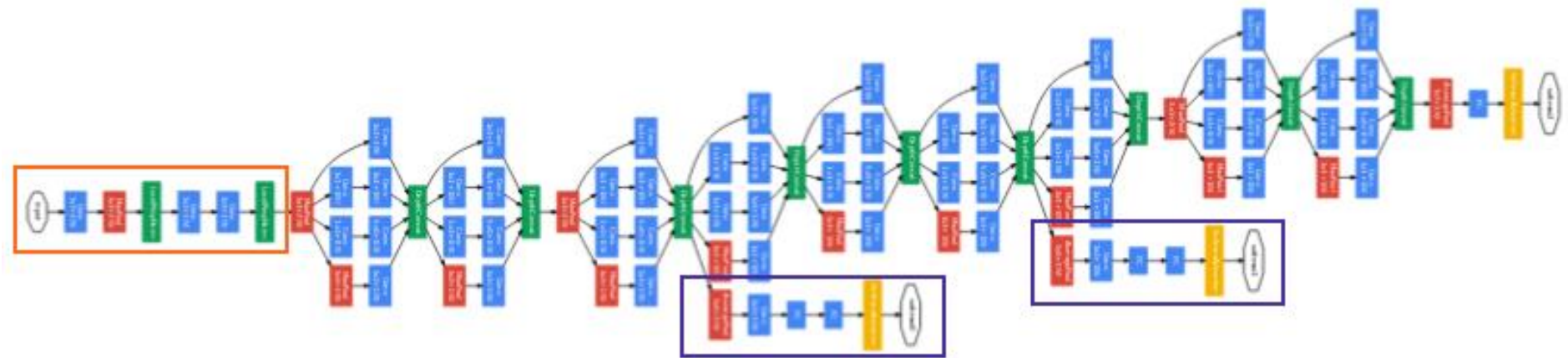
VGG



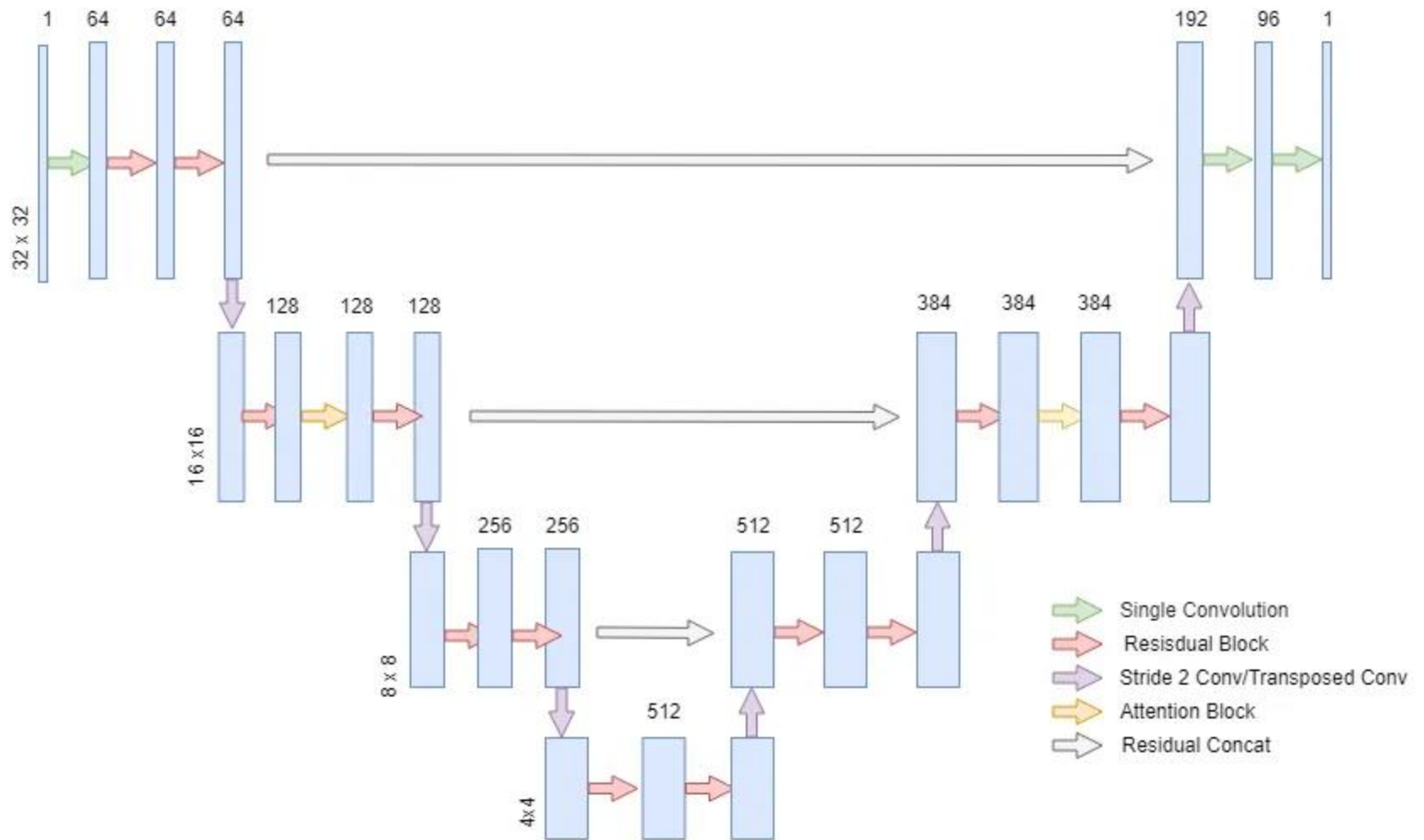
ResNet



Inception V1



U-Net



Predtrénovaný model

- Každá backbone má aj svoje Zoo – zbierku modelov natrénovaných na rôznych datasetoch, napr. na COCO, PascalVOC ...
- Trénovanie siete z predtrénovaného modelu, za ktorý zaradíme svoje (custom) výstupné vrstvy, sa nazýva **transfer learning**

Transfer learning

- Je pravdepodobné, že model na klasifikáciu auta bude v značnej miere podobný modelu pre bicykel
- Preto pri tvorbe druhého môžeme radšej začať od druhého, než na zelenej lúke (from scratch)
- Z modelu natrénovaného pre určitý dataset, napríklad COCO alebo VOC necháme len backbone a ten budeme distribuovať ako **predtrénovaný model**

Fine tuning

- K predtrénovanému modelu pripojíme napr perceptron a trénujeme už len váhy perceptrónu, aby sme získali napr. custom classifier
- Takéto trénovanie je aj oveľa rýchlejšie. Opiera sa však len o hotové features
- Má zmysel preto skúsiť pokračovať v trénovaní, pričom už dovolíme meniť aj váhy backbone. Táto fáza trénovania sa nazýva Fine tuning (jemné doladenie)